

Parallelization in PINNs

Authors- Cody Rucker, Nima Nikkah, Alex Xiao Cai, Quang Dang, Tamanna Saini

Department of Computer Science,
University of Oregon

Outline

Introduction and Theory of PINNs

Neural Network Overview

PDEs

Problem Setup

Manufactured Solution

Multiprocessing and GPU

Parallelizing PINN

Parallelism in Python

Multiprocessing

GPU

DASK

Dask Definition

Implementation

Feed-Forward Network Layer

PINNs can best be understood by first defining a general feed-forward neural network. We let $\mathbf{x} \in \mathbb{R}^n$ (i.e. an n -dimensional vector) and define weights $W \in \mathbb{R}^{m \times n}$ and biases $b \in \mathbb{R}^m$ (i.e. W is an $m \times n$ matrix, and b and n -dimensional vector). A *single hidden layer* of a neural network can be expressed as

$$\ell(\mathbf{x}; \theta) = \phi(W\mathbf{x} + b), \quad \text{where } \theta = (W, b), \quad (1)$$

for activation ϕ , weight W and bias b .

Deep Neural Network

Let $L \in \mathbb{N}_{>0}$ be the network depth and let $\{\phi_i\}_{i=1}^L$ be a collection of activation functions. Take $\mathbf{x} \in \mathbb{R}^n$ to be the network input and network parameters $(W_1, b_1) \in \mathbb{R}^{m \times n} \times \mathbb{R}^m$, $(W_k, b_k) \in \mathbb{R}^{m \times m} \times \mathbb{R}^m$ for $1 < k < L$, and $(W_L, b_L) \in \mathbb{R}^{d \times m} \times \mathbb{R}^d$. Then the recursive composition

$$\ell_0 = \mathbf{x}, \tag{2a}$$

$$\ell_k = \varphi_k(W_k \ell_{k-1} + b_k) \quad \text{for } 0 < k < L \tag{2b}$$

defines a neural network $\mathcal{N} : \mathbb{R}^n \rightarrow \mathbb{R}^d$ (of depth L and width m) with network parameters θ by

$$\mathcal{N}(\mathbf{x}; \theta) = W_L \ell_{L-1} + b_L. \tag{3}$$

General IBVP

Consider the general operator form of an IBVP given by

$$\mathcal{L}[u] = \mathbf{k}, \quad \text{in } \hat{\Omega}, \quad (4a)$$

$$\mathcal{B}[u] = \mathbf{g}, \quad \text{on } \partial\hat{\Omega}, \quad (4b)$$

where $\hat{\Omega} \subseteq \mathbb{R}^n$, with boundary (including internal interfaces, like faults) $\partial\hat{\Omega}$

Learning Problem

$$\mathcal{N}(\mathbf{x}; \theta) \approx u(\mathbf{x})$$

$$\{\mathbf{x}_{\hat{\Omega}}^i, \mathbf{k}^i\}_{i=1}^{N_{\hat{\Omega}}}$$

$$\{\mathbf{x}_{\partial}^i, \mathbf{g}^i\}_{i=1}^{N_{\partial}}$$

$$MSE_{\hat{\Omega}}(\theta) = \frac{1}{N_{\hat{\Omega}}} \sum_{i=1}^{N_{\hat{\Omega}}} |\mathcal{L}[\mathcal{N}](\mathbf{x}_{\hat{\Omega}}^i; \theta) - \mathbf{k}^i|^2 \quad (5a)$$

$$MSE_{\partial}(\theta) = \frac{1}{N_{\partial}} \sum_{i=1}^{N_{\partial}} |\mathcal{B}[\mathcal{N}](\mathbf{x}_{\partial}^i; \theta) - \mathbf{g}^i|^2 \quad (5b)$$

$$MSE(\theta) = MSE_{\hat{\Omega}}(\theta) + MSE_{\partial}(\theta) \quad (5c)$$

$$\theta_{\min} = \arg \min_{\theta} MSE(\theta) \quad (5d)$$

Linear Advection

As an example IBVP we will use the fully-specified linear advection problem given by

$$u_t + u_x = 0 \tag{6a}$$

$$u(x, 0) = u_0(x) \tag{6b}$$

$$u(0, t) = g(t) \tag{6c}$$

Problem Setup

We measure the accuracy of the PINN using the method of manufactured solutions. That is, we used a known solution u^e to derive source terms on the IBVP. Namely take

$$u^e(x, t) = e^{-(x-t)^2} \quad (7)$$

which satisfies the linear advection PDE exactly and defines the initial data u_0 , and boundary data g , to be

$$u_0(x) = e^{-x^2}, \quad (8a)$$

$$g(t) = e^{-t^2}. \quad (8b)$$

MSE equations for our problem

where $\mathcal{N}$ is a trainable network along with objective function MSE given by

$$MSE_{\Omega} = \frac{1}{N_{\Omega}} \sum_{i=1}^{N_{\Omega}} |\tilde{\mathcal{N}}_t + \tilde{\mathcal{N}}_x|^2, \quad (9)$$

$$MSE_{\partial} = \frac{1}{N_{\partial}} \sum_{i=1}^{N_{\partial}} |\tilde{\mathcal{N}} - g|^2, \quad (10)$$

$$MSE = MSE_{\Omega} + MSE_{\partial}. \quad (11)$$

Parallelization of the Problem

Given the application of a domain decomposition, we will use the subnetworks to reconstitute an approximate solution via an orthogonal sum. Using indicator function χ defined on each subdomain $[L_i, L_{i+1}]$ for $i = 1, 2, \dots, K - 1$ as

$$\chi_{[L_i, L_{i+1}]}(x) = \begin{cases} 1 & \text{if } x \in [L_i, L_{i+1}] \\ 0 & \text{otherwise} \end{cases} \quad (12)$$

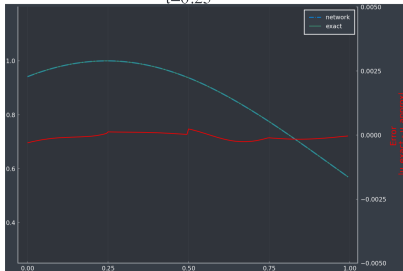
so that the subnetworks are summed together as

$$\tilde{\mathcal{N}}(x, t) = \sum_{i=0}^{K-1} \tilde{\mathcal{N}}_i(x, t) \chi_{[L_i, L_{i+1}]}(x). \quad (13)$$

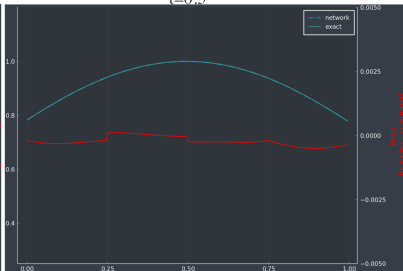


Graphs

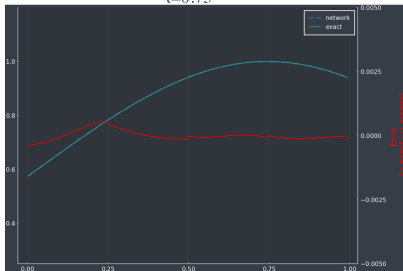
$t=0.25$



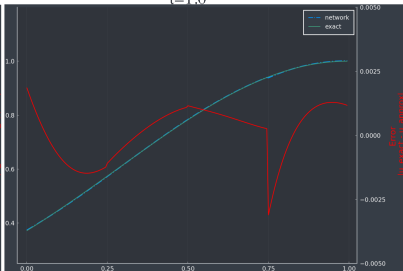
$t=0.5$



$t=0.75$



$t=1.0$



Parallelizing PINN

Algorithm 1 Parallel PINN Training

Input: $E, N, \partial\hat{\Omega}[N], \text{FFNN}[N]$

Output: Trained FFNNs

Initialization: $epoch \leftarrow 0, i \leftarrow 0$

```
1: while  $epoch < E$  do
2:   Do in Parallel
3:   while  $i < N$  do
4:     Train  $\text{FFNN}[i]$  with respect to  $\partial\hat{\Omega}[i]$ 
5:     Update  $\partial\hat{\Omega}[i]$ 
6:      $i \leftarrow i + 1$ 
7:   End Do
8:    $epoch \leftarrow epoch + 1$ 
9: Return Trained Feed-Forward Neural Networks
```

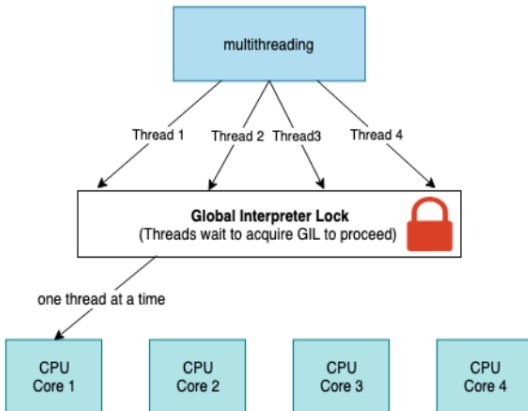
Parallelism in Python

In this project we try implementing our PINN program using different parallel execution models:

- Shared Memory
- Distributed Memory Multiprocessor
- Accelerator Devices

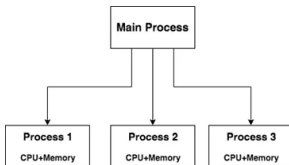
Parallelism in Python

GIL: In CPython implementation of Python, the Global Interpreter Lock prevents multiple threads from executing Python bytecodes at once

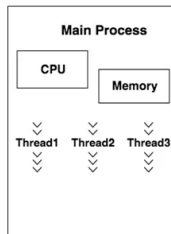


Multiprocessing and Multithreading

Multiprocessing



Multithreading



Multiprocessing

- We tried assigning the training of each neural network instance to a processor
- Sharing States is not suggested for multiprocessing
- Specific data structures such as Array and Value can be shared

Multiprocessing Result

```
process_list = []
for k in networks:
    process_list.append(Process(target=k.train))

for k in process_list:
    k.start()

for k in process_list:
    k.join()
```

```
-----
100%|██████████| 20/20 [00:10<00:00, 1.99it/s]MSE of subnet 0: 9.189e-02
MSE of subnet 1: 1.174e-01
MSE of subnet 2: 7.672e-02
MSE of subnet 3: 9.104e-03
MSE of subnet 4: 1.152e-02
MSE of subnet 5: 1.320e-02
MSE of subnet 6: 4.448e-02
MSE of subnet 7: 6.719e-02
MSE of subnet 8: 9.054e-02
MSE of subnet 9: 1.162e-01
Training run time: 0.460s
-----
Total training time: 9.516s
```

Training with GPU

Running the sequential implementation on a GPU happened to be extremely slow

- On GPU: 269 seconds
- On CPU: 121 seconds

Two solutions are suggested:

- Multiple GPUs (One per NN instance)
- CUDA Context and Memory Management

Dask

Dask is an open-source parallel computing library in Python that is designed to scale and handle large datasets across multiple CPU cores or distributed clusters.

It provides tools for parallelizing computations, scheduling tasks, and managing data structures like arrays, dataframes, and bags.

Dask Key Features

Scalability: Single machines to large clusters

Parallelism: Provides parallel computing capabilities

Integration: NumPy, Pandas, and Scikit-learn

Data Structures: Common data structures like arrays (Dask Array), dataframes (Dask DataFrame), and bags (Dask Bag)

Task Scheduling: Dynamically allocates resources and tasks

Fault Tolerance: Provides fault tolerance mechanisms

Dask Implementation

Delayed: allows users to parallelize and delay the execution of Python functions and computations. (lazy function)

Compute: execute the graph and perform the computations, retrieving the final result in the process.

- Creation of Network and subnetworks

- Initialization of Subnet boundary conditions

- Sampling

- Neighbor boundary update

- Training

Dask Code Snippet

```
1  samples = []
2  for network in networks:
3      local_network = dask.delayed(network)
4      sample = dask.delayed(sample_parallel)(local_network)
5      samples.append(sample)
6
7  data = []
8  for i in range(len(networks)-1):
9      new_data = dask.delayed(new_wrap_data)(samples[i])
10     data.append(new_data)
11
12  neighbor_networks = []
13  for i in range(len(networks)-1):
14      update = dask.delayed(update_bdry)(samples[i+1], data[i])
15      neighbor_networks.append(update)
16
17  train_networks = []
18  for i in range(len(networks)):
19      if i == 0:
20          train_net = dask.delayed(train_parallel)(networks[i])
21      else:
22          train_net = dask.delayed(train_parallel)(neighbor_networks[i-1])
23      train_networks.append(train_net)
24  networks = dask.compute(*train_networks)
```

Figure: Dask code example

Results

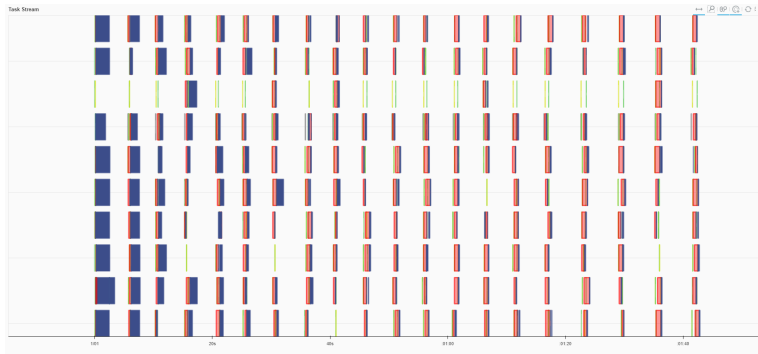


Figure: Dask Task Schedule

Results

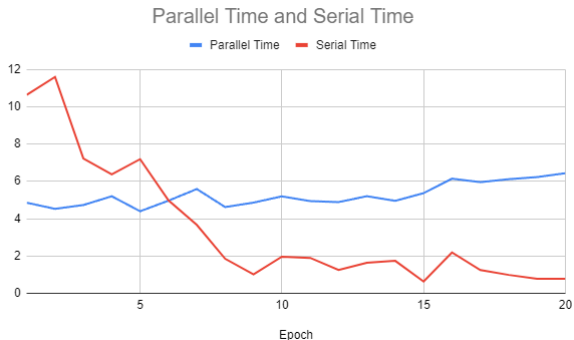


Figure: Dask parallel vs serial

MPI Definition

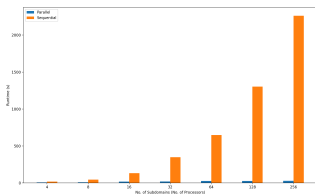
- Message Passing Interface (MPI) is a more traditional standard for parallel computing, offering fine-grained control over distributed computing processes
- It allows for efficient data transfer between different computing nodes, making it well-suited for complex numerical simulations and computational tasks that necessitate a high degree of synchronization between processes

Implementation

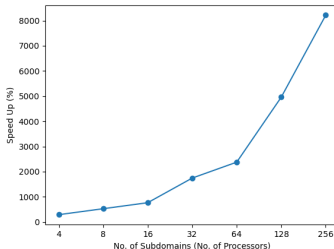
- Use the library mpi4py to implement MPI
- To parallelize our PINN, we divide our problem into a number of equal-sized subdomains and distribute them evenly across multiple processors
- At the end of each training step, the boundary condition of each subdomain is passed to their neighbors, and both training time and message passing time are measured
- After 20 epochs, all PINNs are sent back to the root node (rank 0) and combined using MPI reduce to reconstruct the final solution.

Weak Scaling

For weak scaling, we increase the workload proportionally to the number of processors that we use in each experiment (each processor is assigned to a constant amount of work/processing)



(a)

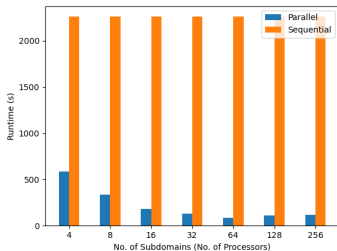


(b)

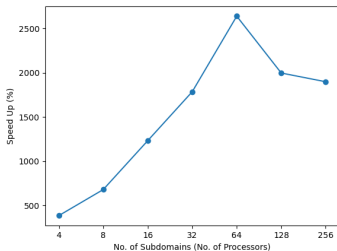
Figure: (a) Execution time compared to the sequential implementation
(b) Speed up compared to the sequential implementation

Strong Scaling

For strong scaling, we divide the domain into 256 subdomains and we keep this number of subdomains fixed regardless of the number of processors that we use



(a)

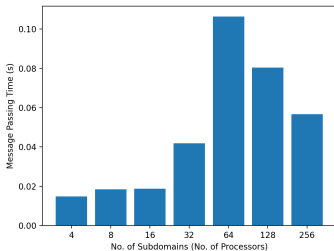


(b)

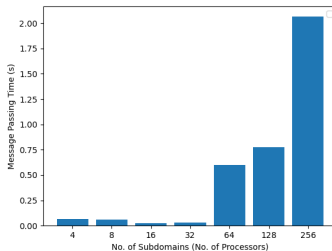
Figure: (a) Execution time compared to sequential implementation
(b) Speed up compared to sequential implementation

Message Passing Time

The message passing time is extremely small and has a negligible effect on the overall runtime for both weak and strong scaling cases.



(a)



(b)

Figure: (a) Message passing time of weak scaling
(b) Message passing time of strong scaling

Weak Scaling vs Strong Scaling code

```
# domain discretization/decomposition
dx = 1 / comm.size
grid = np.arange(0, 1+dx, dx)
sub_domains = [[grid[i], grid[i+1]] for i in range(len(grid)-1)]
subnet = SubNetwork(N_layers, activations, sub_domains[rank], [0.0, 1.0])
```

```
for i in range(20):
    #setup timer
    t0 = time.time()
    commTime = 0
    comm.Barrier()
    start = MPI.Wtime()

    #update domains
    subnet.sample_domains()

    #send and receive boundary data
    if rank < comm.size-1:
        data = subnet.IC.wrap_data()
        commTime -= MPI.Wtime()
        comm.send(data, dest=rank+1)
        commTime += MPI.Wtime()
        commTotal += commTime
    if rank > 0:
        commTime -= MPI.Wtime()
        bdry_data = comm.recv(source=(rank-1))
        commTime += MPI.Wtime()
        commTotal += commTime
    subnet.update_bdry_data(bdry_data)
```

```
#train subnet
subnet.train()
end = MPI.Wtime()
runTime += end-start
t0 = time.time()
```

```
# domain discretization/decomposition
dx = 1 / zsize
domains_per_proc = (zsize // comm.size)
grid = np.arange(0, 1+dx, dx)
sub_domains = [[grid[i], grid[i+1]] for i in range(len(grid)-1)]
subnet = []
for k in range(domains_per_proc):
    subnet.append(SubNetwork(N_layers, activations, sub_domains[rank*domains_per_proc + k], [0.0, 1.0]))
```

```
for i in range(20):
    #setup timer
    t0 = time.time()
    commTime = 0
    comm.Barrier()
    start = MPI.Wtime()

    #update domains
    for k in range(domains_per_proc):
        subnet[k].sample_domains()

    #send and receive boundary data
    for k in range(domains_per_proc):
        if k < domains_per_proc - 1:
            new_data = subnet[k].IC.wrap_data()
            subnet[k+1].update_bdry_data(new_data)
        if rank < comm.size-1:
            data = subnet[domains_per_proc-1].IC.wrap_data()
            commTime -= MPI.Wtime()
            comm.send(data, dest=rank+1)
            commTime += MPI.Wtime()
            commTotal += commTime
        if rank > 0:
            commTime -= MPI.Wtime()
            bdry_data = comm.recv(source=(rank-1))
            commTime += MPI.Wtime()
            commTotal += commTime
            subnet[0].update_bdry_data(bdry_data)
```

```
#train subnets
for k in range(domains_per_proc):
    subnet[k].train()
end = MPI.Wtime()
runTime += end-start
```

Comparison

In this table we have summarized our observations regarding the performance and the accuracy of different parallelization methods that we have addressed in this work and have used to train our neural networks.

Parallelization Method	Performance	Accuracy
GPU	Extremely Slow	High
Multiprocessing	Fast	Extremely Low (Does Not Solve The Equation)
Dask	Moderate Speed Up	High
MPI	Extremely Fast	High

Group Contributions

Tamanna Saini: project management, PINN theory, and model accuracy.

Alex Xiao-Cai: Dask implementation and evaluation.

Quang Dang: Evaluation of MPI with weak scaling and strong scaling methods

Nima Nikkhah: Working with Multiprocessing and torch.cuda implementation of our PINN

Cody Rucker: Problem formulation and theory. MPI implementation.